

# PlantVillage Leaf-Level Disease Detection

## Detailed Methodology, Data Preparation, Transfer Learning, Challenges, and Final Results

Personal Research Notes

September 2026

### 1 Purpose of This Document

This document is a detailed personal research record for the object-detection component of a larger PlantVillage study that also contains classification and segmentation experiments. It is intentionally more detailed than a short conference-paper methodology section. The aim is to preserve what was done, why each step was taken, what failed, what was changed, how the final dataset was constructed, how the transfer-learning experiments were made recoverable, how the models were evaluated, and what limitations must be stated when the work is presented.

### 2 Detection Task Definition

The final task is defined as **whole-leaf localization with PlantVillage disease-class prediction**. Each test image is expected to contain one leaf-level object whose class corresponds to one of the 38 PlantVillage classes.

The task is **not lesion-level disease detection**. The bounding box localizes the leaf as a whole; it does not localize individual infected spots, lesions, fungal regions, or disease symptoms within the leaf.

This distinction is essential for correct reporting.

### 3 Why the Original PlantVillage Dataset Could Not Be Used Directly for Detection

PlantVillage provides class-labeled images and segmented image representations, but the working version used in this study does not provide object-detection bounding-box annotations suitable for YOLO training.

An early exploratory approach created a full-image pseudo box:

$$(x_c, y_c, w, h) = (0.5, 0.5, 1.0, 1.0).$$

Although this allowed YOLO code to run, every ground-truth object occupied the complete image. Therefore, the model was not required to learn meaningful leaf localization. This setup behaved much more like image classification inside a detector framework.

Consequently, the earlier full-image pseudo-box experiments were archived and were **not used as the final detection methodology or final paper results**.

### 4 Key Improvement: Deriving Tight Leaf Boxes from the Segmentation Work

The completed segmentation pipeline had already produced a carefully quality-controlled set of PlantVillage color-segmented pairs. Instead of manually drawing tens of thousands of new boxes,

the final detection pipeline reused this QC-approved supervision to derive tight whole-leaf bounding boxes.

This created a direct methodological connection between the segmentation and detection components while keeping the detection claim limited to whole-leaf localization.

## 5 Source Data Reused from Segmentation

The segmentation stage retained 52,969 QC-approved samples across all 38 PlantVillage classes.

The reused split before detection-specific QC was:

- Training: 42,371
- Validation: 5,300
- Test: 5,298
- Total: 52,969
- Classes: 38

The same split identity was preserved during detection dataset generation instead of creating a new random split.

## 6 Binary Leaf Mask Reconstruction

For each retained segmented RGB image, the same finalized segmentation foreground rule was reused. Let a segmented pixel be

$$\mathbf{x} = (R, G, B).$$

The binary mask was reconstructed as

$$M(\mathbf{x}) = \begin{cases} 1, & \max(R, G, B) > 15, \\ 0, & \max(R, G, B) \leq 15. \end{cases}$$

Thus:

- leaf foreground = 1,
- near-black background = 0,
- threshold = 15.

Where a valid color-segmented pair had different spatial dimensions, the mask was aligned with the color image using nearest-neighbor interpolation. Nearest-neighbor interpolation was retained because it preserves discrete foreground/background labels.

## 7 Tight Bounding-Box Generation

For every binary leaf mask, foreground coordinates were collected:

$$\mathcal{F} = \{(x, y) \mid M(x, y) = 1\}.$$

The axis-aligned leaf bounding rectangle was then defined by:

$$\begin{aligned} x_{\min} &= \min_{(x,y) \in \mathcal{F}} x, & x_{\max} &= \max_{(x,y) \in \mathcal{F}} x + 1, \\ y_{\min} &= \min_{(x,y) \in \mathcal{F}} y, & y_{\max} &= \max_{(x,y) \in \mathcal{F}} y + 1. \end{aligned}$$

The corresponding pixel width and height were:

$$w_b = x_{\max} - x_{\min}, \quad h_b = y_{\max} - y_{\min}.$$

These pixel coordinates were converted to normalized YOLO format:

$$x_c = \frac{x_{\min} + x_{\max}}{2W}, \quad y_c = \frac{y_{\min} + y_{\max}}{2H},$$

$$w = \frac{x_{\max} - x_{\min}}{W}, \quad h = \frac{y_{\max} - y_{\min}}{H},$$

where  $W$  and  $H$  denote image width and height.

Each annotation was stored in the form:

$$\text{class\_id} \quad x_c \quad y_c \quad w \quad h.$$

## 8 Initial Detection-Specific Numerical Quality Control

The first mask-to-box conversion processed all 52,969 source samples.

The initial result was:

- Detection boxes created: 52,969
- Empty/invalid boxes: 0
- Duplicate sample IDs: 0
- Invalid class IDs: 0
- Invalid normalized centers: 0
- Invalid normalized widths/heights: 0
- Classes retained: 38

Numerical validity alone was not considered sufficient. A box can be mathematically valid while still being visually wrong for the intended whole-leaf detection task.

## 9 Suspicious-Box Screening

The box area ratio was defined as:

$$r_A = \frac{w_b h_b}{WH}.$$

Additional width and height ratios were also inspected.

To locate unusually small candidate boxes for manual review, a sample was flagged if at least one of the following was true:

$$r_A < 0.10,$$

$$w < 0.20,$$

or

$$h < 0.15.$$

This rule was used only as a **review trigger**; it was not an automatic exclusion rule.

The screening identified:

- Suspicious candidates: 15
- Percentage of initial candidates: approximately 0.0283%

## 10 Why Suspicious Samples Were Not Automatically Deleted

Some genuinely small or narrow leaves correctly produce small bounding boxes. Therefore, excluding every low-area box would introduce unnecessary bias.

Manual visual review separated two different situations:

1. **Genuine small/narrow leaf:** the visible object itself was small, and the derived box correctly covered the object. These samples were retained.
2. **Invalid partial-leaf box:** the visible leaf was much larger, but the segmentation-derived foreground represented only a small portion of the leaf. Such a box was unsuitable as whole-leaf detection supervision and was excluded.

Manual decisions:

- Suspicious candidates reviewed: 15
- Retained after review: 5
- Excluded after review: 10

The ten exclusions were therefore not random “difficult images.” They were removed because the derived annotation itself was not valid for the final detection task.

## 11 Final Detection Dataset

After detection-specific manual QC, the final dataset contained:

- Training: 42,363
- Validation: 5,300
- Test: 5,296
- Total: 52,959
- Classes: 38

The final source of truth was stored as:

- `final_detection_manifest_v2.csv`
- `detection_class_map.json`
- `detection_preprocessing_config_v2.json`
- `detection_bbox_qc_decisions.csv`
- `detection_bbox_qc_exclusions.csv`

## 12 Complete Detection Workflow Diagram

## 13 Why PlantDoc Was Used

The original PlantVillage images are class-labeled but were not designed as a bounding-box detection dataset. PlantDoc, in contrast, contains real object-detection annotations from plant imagery.

Therefore, PlantDoc was used as an intermediate plant-domain detection stage:

COCO pretrained → PlantDoc → PlantVillage.

The final PlantDoc split used by the notebooks contained:

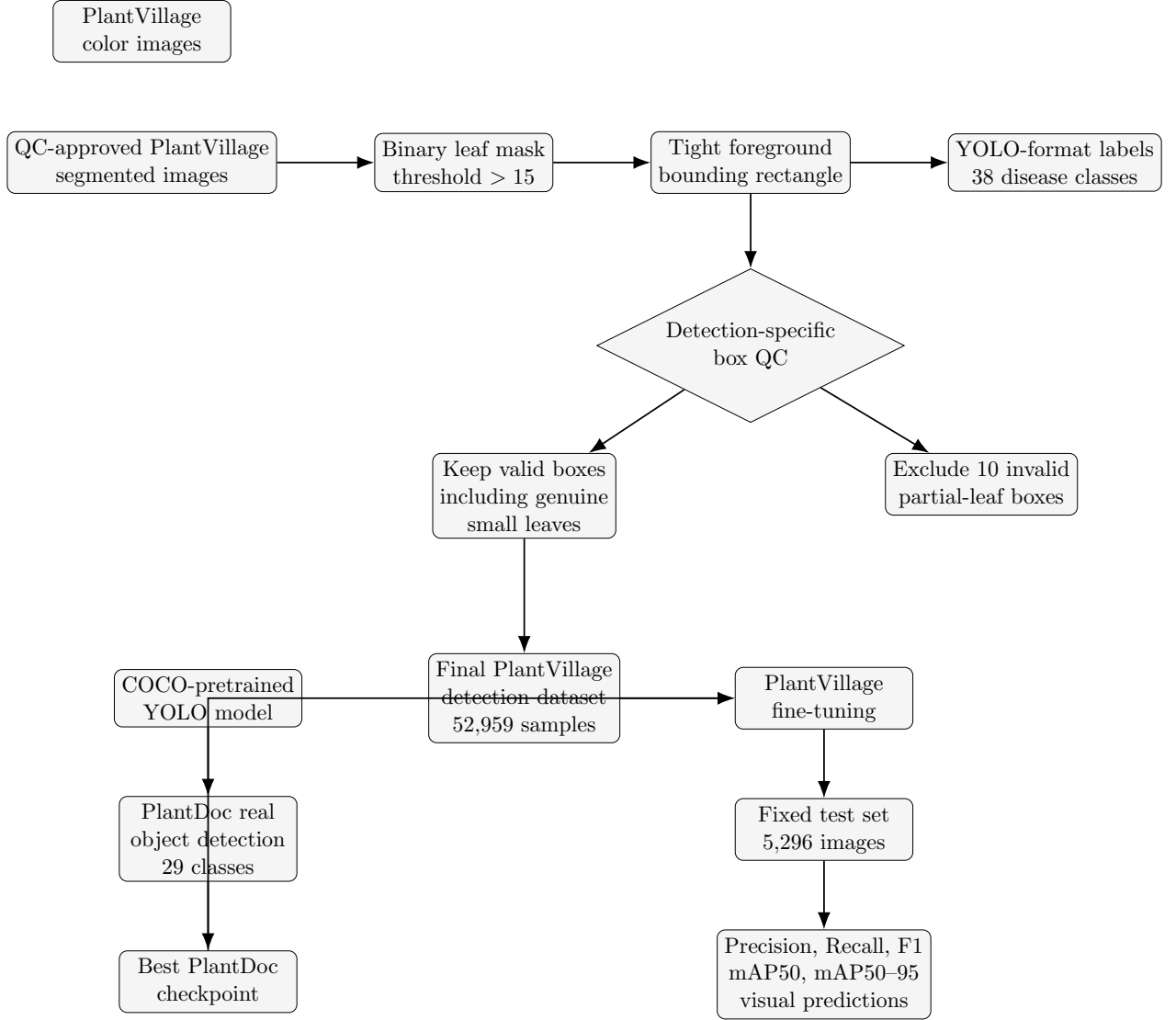


Figure 1: Final leaf-level detection pipeline. The segmented PlantVillage representation is used to derive tight whole-leaf boxes, while PlantDoc provides an intermediate real object-detection domain-adaptation stage before final PlantVillage fine-tuning.

- Training images: 2,335
- Validation images: 236
- Classes: 29

This stage allows a detector initialized from general COCO features to first adapt to real plant-object detection before final fine-tuning on the 38-class PlantVillage leaf-level task.

However, the current study did not run a controlled ablation comparing

COCO  $\rightarrow$  PlantVillage

against

COCO  $\rightarrow$  PlantDoc  $\rightarrow$  PlantVillage.

Therefore, the study should state that PlantDoc was used as an intermediate domain-adaptation stage; it should **not** claim that PlantDoc transfer alone caused a statistically proven performance improvement.

## 14 Why These Three Models Were Selected

### 14.1 YOLOv8n

YOLOv8n was selected as a lightweight nano-scale baseline. It provides a compact detector and establishes whether the task can be solved effectively without a larger model.

### 14.2 YOLOv8s

YOLOv8s was selected as a larger-capacity model from the same YOLOv8 family. Comparing YOLOv8n and YOLOv8s helps evaluate whether additional model capacity improves classification and localization performance under the same dataset and training protocol.

### 14.3 YOLO11n

YOLO11n was selected as a newer nano-scale YOLO-generation alternative. It provides an architecture-generation comparison while remaining relatively lightweight, making it practical for the same Colab/T4 environment.

The three-model design therefore compares:

- a lightweight YOLOv8 baseline,
- a higher-capacity YOLOv8 variant,
- a newer-generation lightweight YOLO detector.

## 15 Common Training Protocol

The final model notebooks use the same two-stage transfer pipeline and common experimental settings.

| Setting                     | Value              |
|-----------------------------|--------------------|
| Image size                  | $320 \times 320$   |
| Batch size                  | 16                 |
| Random seed                 | 42                 |
| Optimizer                   | AdamW              |
| Initial learning rate       | $10^{-3}$          |
| Weight decay                | $5 \times 10^{-4}$ |
| PlantDoc maximum epochs     | 15                 |
| PlantDoc patience           | 5                  |
| PlantVillage maximum epochs | 20                 |
| PlantVillage patience       | 7                  |
| Mixed precision             | Enabled            |
| Deterministic training flag | Enabled            |
| Persistent storage          | Google Drive       |

Ultralytics training-time augmentations and detector defaults remained enabled unless explicitly overridden in the notebooks. The experiment configuration JSON files preserve the explicit settings and software versions.

## 16 Runtime Recovery and Cross-Account Portability

Colab sessions can terminate because of power loss, browser disconnection, GPU limits, or runtime expiry. Therefore, the final detection notebooks were designed to be recoverable.

Each stage saves persistent checkpoints directly to Google Drive:

- **best.pt**: best validation checkpoint,
- **last.pt**: most recent recovery checkpoint.

A completion marker,

`_STAGE_COMPLETE.json`,

is written after a stage finishes.

On a later `Run all`:

1. public datasets are downloaded again to the temporary runtime,
2. the temporary PlantVillage YOLO dataset is rebuilt from the final manifest,
3. the notebook checks Google Drive,
4. a completed stage is skipped,
5. an incomplete stage with **last.pt** is resumed,
6. a stage without a checkpoint starts from its intended initial weights.

The project path is fixed at:

`/content/drive/MyDrive/PlantVillage Detection.`

This makes the workflow portable to another Gmail account if the complete project folder is copied to the same path. To resume a partially trained model, the model-specific `experiments/<MODEL>/` directory must also be copied because it contains **last.pt**.

## 17 Final Test Evaluation

The final model for each architecture is evaluated using the best PlantVillage checkpoint on the same fixed 5,296-image test set.

The following metrics are reported.

### 17.1 Precision

$$Precision = \frac{TP}{TP + FP}.$$

Precision answers: of the detections produced by the model, how many were correct? It penalizes false detections.

### 17.2 Recall

$$Recall = \frac{TP}{TP + FN}.$$

Recall answers: of the ground-truth objects, how many were detected? It penalizes missed leaves.

### 17.3 F1 Score

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}.$$

F1 summarizes the balance between precision and recall and is useful when neither false positives nor false negatives should dominate interpretation.

### 17.4 mAP@0.50

Mean Average Precision at IoU threshold 0.50 evaluates class-wise detection precision/recall while requiring predicted and ground-truth boxes to overlap at least moderately.

It is intuitive and widely reported, but it is relatively lenient because only one IoU threshold is used.

### 17.5 mAP@0.50:0.95

mAP@0.50:0.95 averages Average Precision over multiple IoU thresholds from 0.50 to 0.95. It therefore rewards both correct classification and increasingly precise localization.

For this reason, **mAP@0.50:0.95** was selected as the primary model-ranking metric.

## 18 Qualitative Evaluation

Quantitative metrics were supplemented by visual outputs:

- sampled test predictions with predicted boxes and class labels,
- ground-truth tight box vs. predicted box comparisons,
- precision–recall curves,
- F1 curves,
- precision and recall curves,
- confusion matrices,
- normalized confusion matrices,
- validation label/prediction batches,

- PlantDoc and PlantVillage training curves.

This is important because a high numerical score does not by itself demonstrate that a predicted box is visually appropriate for the intended whole-leaf task.

## 19 Final Results

| Model          | Precision       | Recall          | F1              | mAP50           | mAP50–95        |
|----------------|-----------------|-----------------|-----------------|-----------------|-----------------|
| YOLOv8n        | 0.992832        | 0.991846        | 0.992339        | 0.994014        | 0.981625        |
| <b>YOLOv8s</b> | <b>0.994737</b> | <b>0.995709</b> | <b>0.995223</b> | <b>0.994231</b> | <b>0.983002</b> |
| YOLO11n        | 0.993128        | 0.991207        | 0.992167        | 0.993787        | 0.978832        |

The observed ranking by the primary metric mAP50–95 is:

$$\text{YOLOv8s} > \text{YOLOv8n} > \text{YOLO11n}.$$

YOLOv8s achieved:

- Precision: 0.994737
- Recall: 0.995709
- F1: 0.995223
- mAP50: 0.994231
- mAP50–95: 0.983002

The differences remain small, so the study should describe YOLOv8s as the **best observed model under this experimental setting**, not as universally superior.

## 20 Why the Final Scores Are High

The high test scores are plausible within this experimental setting because:

- PlantVillage imagery is highly controlled compared with field imagery,
- the images usually contain a dominant leaf object,
- the detection boxes were derived from QC-approved leaf masks,
- only ten invalid detection annotations remained after the final detection-specific manual QC,
- the final dataset is large,
- all detectors use pretrained weights and an intermediate plant-domain detection stage,
- the same fixed split and annotation protocol are used across all three final models.

These results should not be interpreted as proof that the same performance will occur on cluttered real-world agricultural scenes.

## 21 Main Challenges Encountered

1. PlantVillage lacked native bounding-box annotations for the intended YOLO experiment.
2. The initial full-image pseudo-box approach allowed YOLO training but did not provide meaningful localization supervision and was rejected from the final methodology.
3. Tight boxes had to be generated from segmentation-derived foreground masks.
4. Numerical box validity did not guarantee semantic validity.
5. Some segmentation-derived masks represented only a small part of a visibly larger leaf, producing misleading whole-leaf boxes.
6. A pure size threshold could not be used for automatic exclusion because some genuinely small leaves also had small but correct boxes.
7. Fifteen suspicious candidates therefore required manual visual review.
8. Five genuine small/narrow objects were retained.
9. Ten invalid partial-leaf annotations were excluded.
10. Colab runtime reliability required persistent checkpointing and auto-resume logic.
11. Multiple Google accounts were used because GPU quotas were limited; fixed Drive-relative paths were therefore required for portability.

## 22 What Was Deliberately Not Claimed

The final experiment does **not** claim:

- lesion-level detection,
- manual bounding-box annotation,
- real-world field generalization,
- statistical proof that PlantDoc transfer outperforms direct COCO-to-PlantVillage fine-tuning,
- universal superiority of YOLOv8s.

Instead, the defensible claim is:

A reproducible leaf-level disease-detection pipeline was constructed by deriving tight whole-leaf boxes from QC-approved PlantVillage segmented images and fine-tuning three YOLO detectors through a common COCO-to-PlantDoc-to-PlantVillage transfer-learning protocol. YOLOv8s achieved the highest observed test mAP50–95 in the final comparison.

## 23 Conference-Presentation Q&A Notes

**Why did you not use the original PlantVillage images directly for YOLO?**

YOLO requires bounding-box supervision. PlantVillage provided class labels and segmented representations but no suitable native object-detection boxes in this workflow. Therefore, tight leaf boxes were derived from the quality-controlled segmented images.

### **Why did you reject the full-image pseudo boxes?**

A full-image box is mathematically valid but does not require the detector to learn localization. Every object has the same spatial extent, so the setup becomes close to classification inside a detection framework. Tight leaf boxes provide meaningful localization supervision.

### **Why did you exclude only ten images?**

After numerical QC, only fifteen boxes were flagged as suspicious. Manual review showed that five were genuine small or narrow leaves with correct boxes. Ten were invalid for whole-leaf detection because the derived box covered only a small portion of a visibly larger leaf.

### **Why not automatically delete all small boxes?**

Because box size alone cannot distinguish a bad partial-leaf annotation from a genuinely small leaf. Automatic deletion would remove valid training samples.

### **Why use PlantDoc?**

PlantDoc supplies real object-detection annotations from plant imagery, so it was used as an intermediate plant-domain adaptation stage before final PlantVillage fine-tuning.

### **Can you prove PlantDoc transfer improved performance?**

Not from the current experiments alone. A direct COCO-to-PlantVillage ablation would be required to isolate that effect. The current study records PlantDoc as an intermediate transfer stage rather than claiming a proven causal improvement.

### **Why is mAP50–95 your main comparison metric?**

Because it averages performance over increasingly strict IoU thresholds and therefore evaluates both correct detection and localization quality more rigorously than mAP50 alone.

### **Why did YOLOv8s perform best?**

YOLOv8s has greater capacity than YOLOv8n and may have benefited from that extra representational capacity on the 38-class task. However, the numerical differences are small, so the result should be presented as the best observed performance in this experiment rather than a universal architectural conclusion.

### **Is this lesion detection?**

No. The box covers the leaf and the detector predicts the PlantVillage disease class. Individual lesions are not annotated or localized.

## **24 Reproducibility Artifacts**

The final detection project stores:

- final detection manifest,
- class mapping,
- preprocessing configuration,
- QC candidate/decision/exclusion records,
- model experiment configuration,

- PlantDoc and PlantVillage training histories,
- best and last model checkpoints,
- test metrics JSON/CSV,
- per-class metrics,
- prediction grids,
- ground-truth vs prediction figures,
- evaluation curves and confusion matrices,
- experiment summaries,
- completion markers supporting auto-resume.

Large `.pt` checkpoints should remain in Google Drive/local backup rather than normal Git tracking.

## 25 Current Remaining Work in the Wider Study

The detection component is now complete. The segmentation component is also complete. The classification component should be reviewed and rerun using the same level of reproducibility and evaluation rigor before the full classification–segmentation–detection paper is finalized.

Only after the classification rerun should the final ICCIT-oriented paper contribution, methodology condensation, results tables, abstract, and conclusion be written.